

EXPERIMENT - 3

Foundation of Deep Learning

Backpropagation from Scratch

Aim

To understand the working of the backpropagation algorithm by performing forward propagation, manually computing gradients, updating network weights using gradient descent, and verifying the calculations using Python.

Objective

To study how a neural network learns by propagating errors backward through the network and updating its weights to minimize the prediction error.

Introduction

Artificial Neural Network learns in such a way that the weights are adjusted continuously until the output produced is nearer to the desired output. In this case, backpropagation is the method of learning where the algorithm calculates the error in the output produced by passing a certain input through the network.

During each training cycle, when a certain input is given to a neural network, the output is calculated by the neural network based on the present set of weights. The difference between the desired output and the produced output is calculated using the cost function. And in case there is an error in the output, it is sent backwards to find out the contribution of each individual weight.

After finding the gradient values in the process of backpropagation, these gradients are used by Gradient Descent to adjust the weights in order to decrease the cost. By doing the process several times, the network is able to learn the mapping between the inputs and outputs.

For this particular experiment, a neural network having 2 inputs, 2 hidden layers, and 1 output is taken into consideration. The calculations are done by hand before verifying the results using python programming language.

Artificial Neural Network (ANN)

An Artificial Neural Network is a type of computer model based on the human brain structure. It involves neurons connected together and processing information using their weighted connections.

A simple artificial neural network includes three layers:

Input layer - Accepts the inputs. Hidden layer - Learns intermediate representations and patterns. Output layer - Returns the output prediction.

Each neuron accepts its inputs, multiplies them with the corresponding weights, adds a bias, and then applies an activation function to return its output.

Forward Propagation

Forward propagation is the first stage of neural network learning. During this stage, the input values are passed through the network layer by layer until the final prediction is obtained.

The weighted sum of a neuron is computed as:

$$z = \sum_{i=1}^n w_i x_i + b$$

where

x_i = Input

w_i = Weight

b = Bias

z = Weighted sum

The weighted sum is then passed through an activation function to obtain the neuron's output.

Sigmoid Activation Function

The sigmoid activation function converts any real-valued input into a value between 0 and 1, making it suitable for binary classification problems.

The sigmoid function is given by:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x) (1 - \sigma(x))$$

The derivative of the sigmoid function is required during backpropagation to compute gradients for updating the weights.

Mean Squared Error (MSE)

The prediction error is measured using the Mean Squared Error (MSE) loss function.

The mathematical expression is:

$$L = \frac{1}{2}(y - \hat{y})^2$$

where

y = Actual output

$\hat{y}$ = Predicted output

A smaller loss value indicates that the prediction is closer to the actual output.

Backpropagation

Backpropagation is the process of propagating the prediction error from the output layer back to the hidden layer. Using the chain rule of calculus, the algorithm calculates the gradient of the loss function with respect to every trainable weight in the network.

The main steps involved are:

- Perform forward propagation.

- Compute the prediction error.
- Calculate the gradient at the output neuron.
- Propagate the error backward to the hidden layer.
- Compute gradients for all weights.
- Update the weights using Gradient Descent.

Backpropagation enables the neural network to improve its predictions after every iteration.

Gradient Descent

Gradient Descent is an optimization algorithm that minimizes the loss function by updating the network weights in the direction opposite to the gradient.

The weight update equation is:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

where

- W = Weight
- η = Learning rate
- L = Loss function
- $\partial W / \partial L$ = Gradient of the loss with respect to the weight

By repeatedly updating the weights, the network gradually reduces the prediction error.

Chain Rule

The chain rule is a mathematical technique used to calculate derivatives of composite functions. In neural networks, it allows the gradients to be propagated from the output layer to the hidden layer.

For the output layer, the gradient is calculated as:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial W}$$

The same principle is applied recursively to compute the gradients of all hidden-layer weights.

Given Data

Input:

$$x = [1, 0]$$

Target Output:

$$y = 1$$

Input-to-Hidden Weight Matrix:

$$W_1 = \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix}$$

Hidden-to-Output Weight Matrix:

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

Activation Function: Sigmoid

Loss Function: Mean Squared Error (MSE)

Learning Rate: $\eta = 0.1$

Manual Calculation:

Manual Calculation

Given Data

- Input: $x = [1, 0]$
- Target Output: $y = 1$
- Input $\rightarrow$ Hidden Weights

$$W_1 = \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix}$$

- Hidden $\rightarrow$ Output Weights

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

- Learning Rate = $\eta = 0.1$
- Activation Function = Sigmoid

Task 1: Hidden Layer Net Input

For Hidden Neuron h_1

$$z_{h1} = (1 \times 0.2) + (0 \times 0.4) = 0.2$$

For Hidden Neuron h_2

$$z_{h2} = (1 \times 0.3) + (0 \times 0.1) = 0.3$$

Task 2: Hidden Layer Output

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$h_1 = \sigma(0.2) = 0.5498$$

$$h_2 = \sigma(0.3) = 0.5744$$

Task 3: Output Layer Net Input

$$\begin{aligned} z_o &= (0.5498 \times 0.5) + (0.5744 \times 0.6) \\ &= 0.2749 + 0.3446 \\ &= 0.6195 \end{aligned}$$

Task 4: Final Output

$$\begin{aligned} \hat{y} &= \sigma(0.6195) \\ &= \frac{1}{1 + e^{-0.6195}} \\ &= 0.6501 \end{aligned}$$

Task 5: Loss

$$\begin{aligned} L &= \frac{1}{2}(y - \hat{y})^2 \\ &= \frac{1}{2}(1 - 0.6501)^2 \\ &= \frac{1}{2}(0.3499)^2 \\ &= \frac{1}{2}(0.1224) \\ &= 0.0612 \end{aligned}$$

Task 6: Output Delta

$$\begin{aligned}\frac{\partial L}{\partial \hat{y}} &= \hat{y} - y \\ &= 0.6501 - 1 = -0.3499\end{aligned}$$

Sigmoid derivative

$$\begin{aligned}\sigma'(x) &= \sigma(x)(1 - \sigma(x)) \\ &= 0.6501(1 - 0.6501) = 0.2275\end{aligned}$$

Output delta

$$\delta_o = (-0.3499)(0.2275) = -0.0796$$

Task 7: Gradient for Hidden → Output

For w_5

$$\begin{aligned}\text{Gradient} &= \delta_o \times h_1 \\ &= (-0.0796)(0.5498) = -0.0438\end{aligned}$$

For w_6

$$\text{Gradient} = (-0.0796)(0.5744) = -0.0457$$

Task 8: Hidden Layer Delta

For h_1

$$\delta_{h1} = (-0.0796)(0.5)(0.5498)(1 - 0.5498) = -0.0098$$

For h_2

$$\delta_{h2} = (-0.0796)(0.6)(0.5744)(1 - 0.5744) = -0.0117$$

Task 9: Gradient for Input → Hidden

For w_{11}

$$\text{Gradient} = (-0.0098)(1) = -0.0098$$

For w_{21}

Since $x_2 = 0$

Gradient = 0

For w_{12}

$$\text{Gradient} = (-0.0117)(1) = -0.0117$$

For w_{22}

Since $x_2 = 0$

Gradient = 0

Task 10: Update Weights

Formula

$$W_{new} = W - \eta \times Gradient$$

Hidden → Output

$$w_5 = 0.5 - 0.1(-0.0438) = 0.5044$$

$$w_6 = 0.6 - 0.1(-0.0457) = 0.6046$$

Input → Hidden

$$w_{11} = 0.2 - 0.1(-0.0098) = 0.2010$$

$$w_{12} = 0.3 - 0.1(-0.0117) = 0.3012$$

$$w_{21} = 0.4$$

$$w_{22} = 0.1$$

Final Updated Weights

Input → Hidden

$$\begin{bmatrix} 0.2010 & 0.3012 \\ 0.4000 & 0.1000 \end{bmatrix}$$

Hidden → Output

$$\begin{bmatrix} 0.5044 \\ 0.6046 \end{bmatrix}$$

Task 11: Verify the Manual Calculations Using Python

```
import numpy as np

# Input
x = np.array([1, 0])

# Target
y = 1

# Input → Hidden Weights
W1 = np.array([
    [0.2, 0.3],
    [0.4, 0.1]
])

# Hidden → Output Weights
W2 = np.array([
    [0.5],
    [0.6]
])

learning_rate = 0.1

# Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Sigmoid Derivative
def sigmoid_derivative(x):
    return x * (1 - x)

# -----
# Forward Propagation
# -----

hidden_input = np.dot(x, W1)
hidden_output = sigmoid(hidden_input)

output_input = np.dot(hidden_output, W2)
predicted_output = sigmoid(output_input)

loss = 0.5 * (y - predicted_output[0])**2

print("Hidden Layer Output")
print(hidden_output)

print("\nPredicted Output")
print(predicted_output)

print("\nLoss")
print(loss)

Hidden Layer Output
[0.549834  0.57444252]

Predicted Output
[0.65012359]

Loss
0.06120675087427858
```

Task 12: Perform Backpropagation

```

# -----
# Backpropagation
# -----

output_delta = (predicted_output - y) * sigmoid_derivative(predicted_output)
hidden_delta = output_delta.dot(W2.T) * sigmoid_derivative(hidden_output)

# Gradients
grad_W2 = hidden_output.reshape(-1,1) * output_delta

grad_W1 = np.outer(x, hidden_delta)

print("Output Delta")
print(output_delta)

print("\nHidden Delta")
print(hidden_delta)

print("\nGradient W2")
print(grad_W2)

print("\nGradient W1")
print(grad_W1)

```

```

... Output Delta
[-0.07958391]

Hidden Delta
[-0.00984917 -0.01167297]

Gradient W2
[[-0.04375794]
 [-0.04571638]]

Gradient W1
[[-0.00984917 -0.01167297]
 [-0.         -0.         ]]

```

Task 13: Update the Weights

```

# -----
# Weight Update
# -----

W2_new = W2 - learning_rate * grad_W2
W1_new = W1 - learning_rate * grad_W1

print("Updated W1")
print(np.round(W1_new,4))

print("\nUpdated W2")
print(np.round(W2_new,4))

```

```

Updated W1
[[0.201  0.3012]
 [0.4    0.1   ]]

Updated W2
[[0.5044]
 [0.6046]]

```

Verification

The Python implementation produced the same hidden layer activations, predicted output, loss value, gradients, and updated weights as obtained from the manual calculations. This verifies the correctness of the forward propagation and backpropagation computations.

Observation

The activation of the hidden layer and prediction of the output in the first step of the neural network were achieved using forward propagation. The prediction error was computed using the Mean Squared Error loss function. In the backpropagation step, the derivatives of loss functions with respect to each weight were calculated using the chain rule. Finally, the weights were updated using gradient descent to minimize the prediction error.

The manual calculations matched the output of the python code.

Result

The experiment successfully demonstrated the implementation of backpropagation in a neural network. Forward propagation produced the predicted output and loss, while backpropagation calculated the gradients required for updating the weights. The updated weights obtained manually matched those generated by the Python program, validating the implementation. This experiment illustrates how neural networks learn by minimizing prediction error through iterative weight updates.

Conclusion

In this experiment, the backpropagation algorithm was investigated using both manual calculation and programming using the Python language. First, the forward propagation method was applied to determine the hidden layer activities, network outputs, and loss from the predictions. Then, the errors were propagated backwards to determine the gradients of the weights using the chain rule and update the weights using gradient descent. The manually calculated values were very similar to the results obtained through the use of the Python language, thus validating the calculations done. Through this experiment, it can be shown that backpropagation is an algorithm that allows neural networks to learn through updating their weights.